

Generic CRUD Dialog Architecture - Specification & Guide

This document outlines the architecture and usage of the generic, asynchronous CRUD (Create, Read, Update, Delete) dialog system built for JavaFX applications using the `CompletableFuture` pattern.

1. Overview

The system uses a **Mixin (Interface with Default Methods)** approach combined with the **Template Method Pattern**. This allows any JavaFX Controller to "inherit" complex Add/Update/Delete workflows simply by implementing an interface and defining the specific configuration steps.

Key Components

- **DynamicFormDialog**: The UI engine that renders forms at runtime.
- **FormField / FieldType**: Configuration objects defining the inputs.
- **AddDialog, UpdateDialog, DeleteDialog**: The logic mixins handling the async flow.
- **WindowManager**: A utility for displaying Alerts.

2. Configuration Classes

2.1 FieldType

An enum defining all supported input widgets.

- **TEXT**: Single-line `TextField`.
- **PASSWORD**: Masked `PasswordField`.
- **TEXTAREA**: Multi-line `TextArea`.
- **NUMBER**: `TextField` (use for Integers).
- **DECIMAL**: `TextField` (use for Doubles).
- **CHECKBOX**: Boolean `CheckBox`.
- **CHOICE**: `ComboBox` (Dropdown).
- **RADIO**: Group of `RadioButtons`.
- **DATE**: `DatePicker`.
- **COLOR**: `ColorPicker`.
- **FILE**: File selection (currently maps to text path).

2.2 FormField

A configuration object representing one row in a form. Use the static factory methods to create instances.

Factory Methods Usage:

```
// Text Field
FormField.text("name", "Full Name", "Default Value");

// Number Field
FormField.integer("age", "Age", 18);
FormField.decimal("price", "Price", 9.99);

// Dropdown (Choice)
List<String> options = List.of("Admin", "User", "Guest");
FormField.choice("role", "User Role", options, "User");

// Radio Buttons
FormField.radio("gender", "Gender", List.of("Male", "Female"), "Male");

// Checkbox
FormField.checkbox("active", "Is Active?", true);

// Read-Only (for IDs)
FormField.readOnly("id", "ID", 101);
```

3. The Dialog Engine: DynamicFormDialog

A utility class that builds a `Dialog<Map<String, String>>` based on a list of `FormFields`.

Usage

It is rarely used directly in Controllers (the Mixins handle it), but can be used standalone:

```
List<FormField> config = List.of(
    FormField.text("username", "Username", ""),
    FormField.password("pass", "Password")
);

DynamicFormDialog dialog = new DynamicFormDialog("Login", config);

dialog.show().ifPresent(results -> {
    String user = results.get("username");
    String pass = results.get("pass");
    // Process input...
});
```

4. The CRUD Mixins (Interfaces)

These interfaces provide the actual business logic. Implement them in your Controller to enable functionality.

4.1 AddDialog<E>

Handles the creation of new entities.

Generic Type: <E> = The Entity Class (e.g., Client).

Methods to Implement:

1. List<FormField> getAddFormConfig(): Return the list of fields for the "Add" form.
2. E createEntity(Map<String, String> results): Convert the form output (String map) into a new Entity object.

Execution:

Call executeAdd() in your @FXML handler.

```
@FXML
public void handleAdd() {
    executeAdd(
        "Add New Client",    // Title
        service::add,        // Async Save Method (Function<E, CompletableFuture<E>>)
        this::loadData       // Refresh Method (Runnable)
    );
}
```

4.2 UpdateDialog<ID, E, DTO>

Handles the editing of existing entities. fetching the latest version from the DB before editing to prevent stale data overwrites.

Generic Types:

- <ID>: ID type (e.g., Integer).
- <E>: Entity type (e.g., Client).
- <DTO>: Table Row Object (e.g., ClientDto).


```
}
```

5. Implementation Guide: Creating a Controller

To create a fully functional CRUD controller (ClientController), follow these steps:

Step 1: Extend Abstract & Implement Interfaces

```
public class ClientController
    extends AbstractPagingTableViewController<Integer, Client, ClientDto, ClientFilter>
    implements AddDialog<Client>,
               UpdateDialog<Integer, Client, ClientDto>,
               DeleteDialog<Integer, ClientDto> {
```

Step 2: Implement Template Methods

Define the form structure and mapping logic.

```
// --- ADD IMPL ---
@Override
public List<FormField> getAddFormConfig() {
    return List.of(
        FormField.text("name", "Name", ""),
        FormField.decimal("budget", "Budget", 0.0)
    );
}

@Override
public Client createEntity(Map<String, String> results) {
    return new Client(
        results.get("name"),
        Double.parseDouble(results.get("budget"))
    );
}

// --- UPDATE IMPL ---
@Override
public List<FormField> getUpdateFormConfig(Client entity) {
    return List.of(
        FormField.readOnly("id", "ID", entity.getId()),
```

```

        FormField.text("name", "Name", entity.getName()),
        FormField.decimal("budget", "Budget", entity.getBudget())
    );
}

@Override
public void updateEntity(Client entity, Map<String, String> results) {
    entity.setName(results.get("name"));
    entity.setBudget(Double.parseDouble(results.get("budget")));
}

// --- DELETE IMPL ---
@Override
public String getNameForDelete(ClientDto dto) {
    return dto.getName();
}

```

Step 3: Connect FXML Actions

Wire up the buttons to the execution methods.

```

@FXML
public void onAdd() {
    executeAdd("Add Client", service::add, this::loadData);
}

@FXML
public void onUpdate() {
    executeUpdate(
        tableView.getSelectionModel().getSelectedItem(),
        ClientDto::getId,
        service::findOne,
        service::update,
        this::loadData
    );
}

@FXML
public void onDelete() {
    executeDelete(
        tableView.getSelectionModel().getSelectedItem(),

```

```

        ClientDto::getId,
        service::delete,
        this::loadData
    );
}

```

6. Asynchronous Considerations

All interfaces are designed for **Non-Blocking UI**.

- **Service Requirements:** Your Service methods (add, update, delete, findOne) **must** return `CompletableFuture`.
- **Threading:**
 - Form rendering and User Input happen on the **JavaFX Application Thread**.
 - Database operations happen on a **Background Thread**.

7. Controller Specification Guide

This section outlines how to implement new controllers using the abstract base classes. This guide details the requirements, methods to implement, and FXML structure needed for each type of view.

7.1 Simple Table View Specification

Base Class: `AbstractTableViewController<ID, E, DTO>`

Use Case: Displaying a small, non-paginated dataset in a table (e.g., a "Dashboard" widget or a lookup table).

- **Required FXML Elements:**
 - `TableView<DTO> fx:id="tableView"`: The main table component.
- **Constructor:** Default (no-args) constructor is usually sufficient.
- **Methods to Implement:**
 1. `Function<E, DTO> getDtoMapper()`: Return a reference to your DTO constructor (e.g., `ClientDto::new`).
 2. `void initialize()` (Optional but recommended): Override to set up `TableColumn` cell value factories. **Must call `super.initialize()`**.
- **Setup Call:**
 - Call `setBaseService(service)` to inject the service and trigger the initial `loadData()`.

7.2 Paged Table View Specification

Base Class: `AbstractPagingTableViewController<ID, E, DTO, Filter>`

Use Case: Displaying large datasets with navigation controls (Next/Previous) and filtering

capabilities (e.g., Main Client Management View).

- **Required FXML Elements:**
 - TableView<DTO> fx:id="tableView"
 - Button fx:id="buttonPrevious"
 - Button fx:id="buttonNext"
 - Label fx:id="labelPage"
- **Constructor:**
 - Must call super(currentPage, pageSize, totalElements, new Filter()).
 - Example: super(0, 5, 0, new ClientFilter()).
- **Methods to Implement:**
 1. Function<E, DTO> getDtoMapper(): Return the DTO mapper.
 2. void initialize(): Override to configure columns and listeners. **Must call super.initialize().**
- **Setup Call:**
 - Call setBaseService(service) (or a custom setService wrapper) to start the observer and load data.

7.3 Simple List View Specification

Base Class: AbstractListViewController<ID, E, DTO>

Use Case: A simple vertical list of items, often with custom graphics per row (e.g., a task list or notification feed).

- **Required FXML Elements:**
 - ListView<DTO> fx:id="listView"
- **Methods to Implement:**
 1. Function<E, DTO> getDtoMapper(): Return the DTO mapper.
 2. void setupListCellFactory(): Define how each item looks. This is where you create custom ListCell implementations with graphics (Checkboxes, Labels).
- **Setup Call:**
 - Call setBaseService(service) to initialize.

7.4 Paged List View Specification

Base Class: AbstractPagingListViewController<ID, E, DTO, Filter>

Use Case: A custom graphical list (cards, tasks) that requires pagination to manage performance.

- **Required FXML Elements:**
 - ListView<DTO> fx:id="listView"
 - Button fx:id="buttonPrevious"
 - Button fx:id="buttonNext"
 - Label fx:id="labelPage"
- **Constructor:**
 - Must call super(startPage, pageSize, totalDocs, new Filter()).

- **Methods to Implement:**
 1. `Function<E, DTO> getDtoMapper():` Return the DTO mapper.
 2. `void setupListCellFactory():` Define the ListCell logic (see below).
- **Setup Call:**
 - Call `setBaseService(service)` to initialize.

7.5 Implementation Guide: Custom Rows & Interactive Cells

To implement the "Manager" features (Coloring, CheckBoxes, ComboBoxes), apply the following logic inside your implementations:

A. Interactive List Cells (for ListViews)

Inside `setupListCellFactory()`:

1. **Define Controls:** Create `CheckBox`, `ComboBox`, etc., inside the `ListCell` constructor or field declaration to reuse them (Performance optimization).
2. **Add Listeners:** Attach `setOnAction` or `addListener` to these controls.
 - *Async Update:* Call `service.update()` inside the listener for database changes (e.g., `ComboBox`).
 - *Local Update:* Update a local `Set<ID>` or style for transient changes (e.g., `CheckBox`).
3. **Override `updateItem`:**
 - Bind the DTO values to the controls (`setText`, `setValue`).
 - Call a helper method (e.g., `updateRowColor`) to change `setStyle()` based on DTO state.

B. Row Coloring (for TableViews)

Inside `initialize()`:

1. **Set Row Factory:** Use `tableView.setRowFactory(...)`.
2. **Override `updateItem`:** Check the item's properties (or local state) and apply CSS styles:


```
row.setStyle("-fx-background-color: lightgreen;"); // Completed
row.setStyle("-fx-background-color: gold;");    // VIP
```

See `ManagerVipTableViewPagingController.java` for the complete pattern.

C. Custom Table Cells (for TableViews)

Inside `initialize()`:

1. **Set Cell Factory:** Use `col.setCellFactory(...)` on the specific `TableColumn`.
2. **Create Graphic:** Instantiate the control (`CheckBox/ComboBox`).
3. **Handle Events:** Add logic to handle user interaction (saving to DB or updating UI).
4. **Set Graphic:** Use `setGraphic(myControl)` inside `updateItem`.